

AI Does Not Help Programmers

By Bertrand Meyer

Posted Jun 3 2023

Everyone is blown away by the new AI-based assistants. (Myself included: see [an earlier article](#) on this blog which, by the way, I would write differently today.) They pass bar exams and write songs. They also produce programs. Starting with Matt Welsh's [article](#) in *Communications of the ACM*, many people now pronounce programming dead, more recently *The New York Times*.

I have tried to understand how I could use ChatGPT for programming and, unlike Welsh, found almost nothing. If the idea is to write some sort of program from scratch, well, that's yes. I am willing to believe the experiment [reported on Twitter](#) of how a beginner used Copilot to beat hands-down a professional programmer for a from-scratch development of a Minimum Viable Product program, from "Figma screens and a set of specs." I have also seen people who know next to nothing about programming get a useful program prototype by just typing in a general specification. I am talking about something else, the kind of use that Welsh touts: a professional programmer using an AI assistant to do a better job. It doesn't work.

Precautionary observations:

- *Caveat 1:* We are in the early days of the technology and it is easy to mistake teething problems for fundamental limitations. (*PC Magazine's* [initial review](#) of the iPhone: "it's just a plain lousy phone, and although it makes some exciting advances in handheld Web browsing it is not the Internet in your pocket.") Still, we have to assess what we have, not what we could get.
- *Caveat 2:* I am using ChatGPT (version 4). Other tools may perform better.
- *Caveat 3:* It has become fair game to try out ChatGPT or Bard, etc., into giving wrong answers. We all have great fun when they tell us that Famous Computer Scientist X has received the Turing Award and next (equally wrongly) that X is dead. Such exercises have their use, but here I am doing something different: not trying to trick an AI assistant by pushing it to the limits of its knowledge, but genuinely trying to get help from it for my key purpose, programming. I would love to get correct answers and, when I started, thought I would. What I found through honest, open-minded enquiry is at complete odds with the hype.

- *Caveat 4:* The title of this article is rather assertive. Take it as a proposition to be debated (“This house believes that...”). I would be interested to be proven wrong. The main immediate goal is not to edict an inflexible opinion (there is enough of that on social networks), but to spur a fruitful discussion to advance our understanding beyond the “Wow!” effect.

Here is my experience so far. As a programmer, I know where to go to solve a problem. But I am fallible; I would love to have an assistant who keeps me in check, alerting me to pitfalls and correcting me when I err. A effective pair-programmer. But that is not what I get. Instead, I have the equivalent of a cocky graduate student, smart and widely read, also polite and quick to apologize, but thoroughly, invariably, sloppy and unreliable. I have little use for such supposed help.

It is easy to see how generative AI tools can perform an excellent job and outperform people in many areas: where we need a result that comes very quickly, is convincing, resembles what a top expert would produce, and is almost right on substance. Marketing brochures. Translations of Web sites. Actually, translations in general (I would not encourage anyone to embrace a career as interpreter right now). Medical image analysis. There are undoubtedly many more. But programming has a distinctive requirement: programs must be right. We tolerate bugs, but the core functionality must be correct. If the customer’s order is to buy 100 shares of Microsoft and sell 50 of Amazon, the program should not do the reverse because an object was shared rather than replicated. That is the kind of serious error professional programmers make and for which they need help.

AI in its modern form, however, does not generate correct programs: it generates programs inferred from many earlier programs it has seen. These programs look correct but have no guarantee of correctness. (I am talking about “modern” AI to distinguish it from the earlier kind—largely considered to have failed—which tried to reproduce human logical thinking, for example through expert systems. Today’s AI works by statistical inference.)

Fascinating as they are, AI assistants are not works of logic; they are works of words. Large *language* models: smooth talkers (like the ones who got all the dates in high school). They have become incredibly good at producing text that looks right. For many applications that is enough. Not for programming.

Some time ago, I published on this blog a sequence of articles that tackled the (supposedly) elementary problem of binary search, each looking good and each proposing a version which, up to the last installments, was wrong. (The first article is [here](#); it links to its successor, as all items in the series do. There is also [a version on my personal blog](#) as a single article, which may be more convenient to read.)

I submitted the initial version to ChatGPT. (The interaction took place late May; I have not run it again since.)

CH

Here is an attempt at binary search. Can you tell me if it is correct?

from

$i := 1; j := n$ — Result initialized to 0.

until $i = j$ loop

$m := (i + j) // 2$ — Integer division

if $t[m] \leq x$ then $i := m$ else $j := m$ end

end

if $x = t[i]$ then Result := i end

The answer begins with a useful description of the problem:



The binary search algorithm you've given is nearly correct, but it has a small issue that could lead to an infinite loop in certain cases.



When the search space is narrowed down to two elements, if the ' x ' is less than ' $t[m]$ ', you set ' j ' to ' m ', which would still include ' m ' in the search space for the next iteration. However, you've already checked ' $t[m]$ ' and know that ' x ' is not equal to ' $t[m]$ '. Therefore, you should exclude ' m ' from the search space by setting ' j ' to ' $m - 1$ ' in the '**else**' branch.

Good analysis; similar in fact to the debunking of the first version in [my own follow-up](#). The problem can actually arise with any number of elements, not just two, but to prove a program incorrect it suffices to exhibit a single counterexample. (To prove it correct, you have to show that it works for all examples.) But here is what ChatGPT comes up with next, even though all I had actually asked was whether the program was correct, not how to fix it:

Here's the corrected version of your algorithm:

eiffel

Copy code

```

from
  i := 1 ; j := n
until
  i > j
loop
  m := (i + j) // 2
  if t [m] ≤ x then
    i := m + 1
  else
    j := m - 1
  end
end
if x = t [i - 1] then
  Result := i - 1
end

```

(Please examine this code now!) It includes helpful comments:

In this corrected version, when ' x ' is less than ' $t[m]$ ', we set ' j ' to ' $m - 1$ ', effectively excluding ' m ' from the search space for the next iteration. Similarly, if ' $t[m]$ ' is less than or equal to ' x ', we set ' i ' to ' $m + 1$ '. This ensures that we don't get stuck in an infinite loop when ' i ' and ' j ' are consecutive.

Please note that this binary search algorithm assumes that the array ' t ' is sorted in ascending order, which is the standard prerequisite for binary search. Also, the array indices start from 1 in this case, which is common in Eiffel but different from many other languages where array indices start from 0.

All this is very good, but if you did examine the proposed replacement code, you may have found something fishy, as I did.

I report it:

 What happens in your revised version with a single element `t[1] = 1` (i.e. `n = 1` as well) and a search value `x = 0`? Isn't it the case that after the loop we try to access `t[0]`?

Indeed, in trying to fix my bug, ChatGPT produced another buggy version, although the bug is a new one. There is an eerie similarity with my own original sequence of binary search posts, where each attempt introduced a version that seemed to correct the mistake in the preceding one —only to reveal another problem.

The difference, of course, is that my articles were pedagogical, instead of asserting with undaunted assurance that the latest version is the correct fix!

One thing ChatGPT is very good at is apologizing:

 You are correct, and I apologize for the oversight. The edge case where the array has only one element and the searched value is less than this element wasn't properly handled.

In the given case, the loop would indeed terminate with `'i'` set to 1 and `'j'` set to 0, and the last line would attempt to access `'t[i - 1]'` or `'t[0]'`, which is not valid in an Eiffel array indexed from 1.

Well, for my part, when looking for an assistant I am all for him/her/it to be polite and to apologize, but what I really want is that the assistant be *right*. Am I asking too much? ChatGPT volunteers, as usual, the corrected version that I had not even (or not yet) requested:

We can fix this by checking the final value of `'i'` before attempting to access the array:

```
eiffel
from
  i := 1 ; j := n
until
  i > j
loop
  m := (i + j) // 2
  if t[m] < x then
    i := m + 1
  else
    j := m - 1
  end
end
if i > 1 and then x = t[i - 1] then
  Result := i - 1
end
```

Now, we only access `'t[i - 1]'` if `'i'` is greater than 1. So in the case where `'n = 1'` and `'x = 0'`, we won't attempt to access `'t[0]'`. The `'and then'` operator in Eiffel ensures that the second condition (`'x = t[i - 1]'`) is not evaluated if the first one (`'i > 1'`) is false, thus preventing the out-of-bounds access.

Again, I apologize for the initial oversight, and thank you for your careful reading and questioning!

(Do you also find that the tool doth apologize too much? I know I am being unfair, but I cannot help think of the French phrase *trop poli pour être honnête*, too polite to be honest.)

At this point, I did not even try to determine whether that newest version is correct; any competent programmer knows that spotting cases that do not work and adding a specific fix for each is not the best path to a correct program.

I, too, remain (fairly) polite:

CH OK, but this looks like a kludge designed to address just this one counterexample. How do I know that your revised version is now correct?

Now I am in for a good case of *touché*: ChatGPT is about to lecture me on the concept of loop invariant!

 You're correct in wanting a more systematic approach to verifying the correctness of the algorithm. One way to approach this is through the method of loop invariants, a concept in computer science used to help reason about the correctness of algorithms.

A loop invariant is a condition that is initially true and remains true after each iteration of a loop. By identifying a suitable loop invariant and proving that it holds, we can establish the correctness of an algorithm.

For the binary search algorithm, a possible loop invariant is:

- If 'x' is in the array 't', then it's within the subarray 't[i..j]'.

This invariant holds initially because 'i' and 'j' are set to the bounds of the array. On each iteration of the loop, we halve the search space by adjusting either 'i' or 'j' based on the comparison between 'x' and 't[m]'. This ensures that if 'x' is in the array, it remains within the subarray 't[i..j]'.

When the loop terminates ('i > j'), if 'x' was in the array, it must be at 't[i-1]', because 'i' would have been incremented past it. We can check for this by comparing 'x' to 't[i-1]', but only if 'i > 1' (to avoid accessing 't[0]'). This check is safe because 'i' can only be less than or equal to 1 if 'j' is 0, and 'j' can only be 0 if the array was empty or 'x' was less than every element in the array.

While this reasoning gives us confidence in the correctness of the algorithm, it's always a good idea to test the implementation with a variety of inputs to confirm it behaves as expected.

I never said or implied, by the way, that I “*want a more systematic way of verifying the correctness of the algorithm.*” Actually, I do, but I never used words like “*systematic*” or “*verify*.” A beautiful case of mind-reading by statistical inference from a large corpus: probably, people who start whining about remaining bugs and criticize software changes as “kludges” are correctness nuts like me who, in the next breath, are going to start asking for a systematic approach and verification.

I am, however, a tougher nut to crack than what my sweet-talking assistant—the one who is happy to toss in knowledge about fancy topics such as class invariant—thinks. My retort:

CH I don't understand this invariant. More precisely I understand it of course, but it only makes sense if i..j is a sub-interval of the array's index range, i.e. 1..n. Shouldn't the invariant also say that 1 <= i, j <= n?

There I get a nice answer, almost as if (you see my usual conceit) the training set had included our [loop invariant survey](#) (written with Carlo Furia and Sergey Velder) in ACM's *Computing Surveys*. Starting with a bit of flattery, which can never hurt:



I appreciate your precision and patience. Indeed, we should start with the condition on `'i'` and `'j'` before proceeding to the condition involving `'x'` and `'t'`. It's also true that proving correctness through loop invariants should ideally lead us to a solid understanding of our algorithm, reducing the necessity for extensive testing.

Let's revise the loop invariant accordingly:

• `'1' <= i <= j <= n'` and then if `'x'` is in the array `'t'`, then it's within the subarray `'t[i..j]'`.

This expresses that `'i'` and `'j'` are within the valid bounds of the array, and if `'x'` is in `'t'`, then it's in the subarray from `'i'` to `'j'`. This invariant holds before the loop and is maintained by each iteration. Once the loop terminates, we can conclude that `'x'` is not in the subarray `'t[i..j]'` because `'i' > j'`. Thus, if `'i' > 1'`, we can safely check `'t[i-1]'` for `'x'`.

I hope this gives a more rigorous justification for the correctness of the algorithm. Your questions are indeed helpful in refining the understanding and explanation of these important concepts. Thank you.

And then I stopped.

Not that I had succumbed to the flattery. In fact, I would have no idea where to go next. What use do I have for a sloppy assistant? I can be sloppy just by myself, thanks, and an assistant who is even more sloppy than I is not welcome. The basic quality that I would expect from a supposedly intelligent assistant—any other is insignificant in comparison—is to be *right*.

It is also the only quality that the ChatGPT class of automated assistants cannot promise.

Help me produce a basic framework for a program that will “kind-of” do the job, including in a programming language that I do not know well? By all means. There is a market for that. But help produce a program that has to work correctly? In the current state of the technology, there is no way it can do that.

For software engineering there is, however, good news. For all the hype about not having to write programs, we cannot forget that any programmer, human or automatic, needs specifications, and that any candidate program requires verification. Past the “Wow!”, stakeholders eventually realize that an impressive program written at the push of a button does not have much use, and can even be harmful, if it does not do the right things—what the stakeholders want. (The requirements literature, including [my own recent book on the topic](#), is there to help us build systems that achieve that goal.)

There is no absolute reason why Generative AI For Programming could not integrate these concerns. I would venture that if it is to be effective for serious professional programming, it will have to spark a wonderful renaissance of studies and tools in formal specification and verification.

Bertrand Meyer is a professor and Provost at the Constructor Institute (Schaffhausen, Switzerland) and chief technology officer of Eiffel Software (Goleta, CA).

Submit an Article to CACM

CACM welcomes unsolicited submissions on topics of relevance and value to the computing community.

Join the Discussion (0)